GPT (Generative Pre-trained Transformer)

A Research Report on a Transformer-Based Model

1. Architecture Overview

GPT stands for Generative Pre-trained Transformer. OpenAI introduced it in 2018, building it from the decoder half of the Transformer architecture first described in the 2017 paper "Attention Is All You Need." BERT uses the encoder half and reads in both directions at once. GPT only uses the decoder, and reads left to right.

A GPT model is a stack of decoder blocks, each one containing masked multi-head self-attention followed by a feed-forward layer, with residual connections and layer normalisation around both. The masking matters: when GPT processes a word, it can only look at words that came before it, never after. A triangular attention mask blocks any position from seeing future tokens. That restriction is what forces GPT to generate text in order, one word after another, instead of seeing a full sentence at once the way BERT does.

Text has to go through an embedding layer before it reaches the decoder blocks at all. Each word, or more precisely each subword token, becomes a dense vector of several hundred or several thousand numbers. Then positional encoding adds back information about word order, since the Transformer processes every position in parallel and has no built-in sense of sequence. Without that step, "dog bites man" and "man bites dog" would look identical to the model same words, same embeddings, just rearranged.

Multi-head self-attention is the part that actually gives GPT context. Instead of computing one attention pattern, the model splits the calculation into several parallel heads, and each head can end up tracking a different kind of relationship. One might end up tracking subject-verb agreement, another which pronoun goes with which noun, another general topic continuity across a paragraph. The heads' outputs get combined and run through the feed-forward layer before moving to the next block in the stack.

The scaling across versions is worth noting on its own. GPT-1 (2018) had 117 million parameters across 12 decoder layers. GPT-2 (2019) went up to 1.5 billion. GPT-3 (2020) reached 175 billion and showed that scale by itself could produce abilities like few-shot learning that smaller models simply didn't have. GPT-4 (2023) is believed to use a mixture-of-experts setup with parameters in the trillions, where only part of the network activates for any given input a way of staying efficient despite the larger total size. The architecture itself hasn't changed much across all this. What changed is size, training data, and what happens after pre-training.

2. Working Principle

GPT is autoregressive. It predicts the next word given everything written so far, appends that word to the input, and feeds the whole thing back through the model to predict the word after that. This repeats one token at a time until it hits a stop condition a maximum length, or an end-of-text marker.

At every position, GPT produces a probability for every word in its vocabulary through a Softmax function applied to the final decoder layer's output. Picking the single highest-probability word is possible, but most GPT applications don't do that they sample instead. Temperature controls how safe or how random the choice is: low temperature sticks close to the likely words and gets repetitive, high temperature lets in less likely words and gets more varied but sometimes less coherent. Top-k and top-p sampling narrow the field to plausible candidates so the model doesn't occasionally pick something nonsensical.

Training happens in two stages. Pre-training is where the model reads huge amounts of text and learns to predict the next word with no human labelling at all. That gives it a broad statistical sense of grammar and facts and style, but on its own it doesn't make the model particularly safe or helpful to talk to. The second stage, used in something like ChatGPT, is fine-tuning combined with Reinforcement Learning from Human Feedback. Human reviewers rate several responses to the same prompt, those ratings train a reward model, and GPT gets adjusted to score better against that reward model. In practice this means the model ends up producing answers that are actually useful rather than just statistically common.

This is the self-attention idea from the Week 8 lesson in practice: every word GPT generates is shaped by an attention-weighted look at everything before it, which is how it keeps a long passage on-topic and grammatically consistent. It's also why GPT loses track of earlier details in a long enough conversation once you go past the context window, the earliest tokens simply aren't visible to attention anymore.

3. Advantages

GPT writes fluently across pretty much any register, from a formal report to casual conversation, usually without an obvious seam between the prompt and what it generates next.

Larger versions can pick up a new task from just a few examples in the prompt, sometimes from none at all, just a plain instruction. That cuts out the need to train a separate model for every new task, which used to be a real bottleneck in NLP.

One model handles essays, question answering, summarising, translation, and code, without a different architecture for each. It effectively replaces a pile of specialised systems that used to be trained independently.

Because it's built on the Transformer, GPT trains using parallel processing across the whole input sequence, not one word at a time the way RNNs and LSTMs have to. That's a big part of why it's been possible to scale these models to their current size in a reasonable amount of training time.

4. Limitations

GPT can sound completely confident while being factually wrong, because it's predicting plausible next words from training patterns, not checking facts against any source in real time. In something like medicine, law, or finance, a wrong answer that sounds right is genuinely dangerous.

Running and training large GPT models takes serious compute GPUs or TPUs, not a laptop. That cost also limits how often these models get retrained on newer data, which is part of why their knowledge goes stale.

GPT writes one word at a time with no real plan for the whole response before it starts. So it can drift off topic, contradict something it said three paragraphs earlier, or struggle with reasoning that needs to stay consistent across a long answer.

Training on huge amounts of internet text means GPT picks up whatever social, cultural, or factual bias is sitting in that text. Fine-tuning and RLHF help, but they don't get rid of it completely the bias comes from the scale of pre-training, not just the adjustment stage afterward.

5. Real-World Applications

GPT-based models show up in a lot of everyday and professional tools at this point. A few of the major ones:

Application	**Example**
Chatbots and virtual assistants	ChatGPT and customer support tools that hold multi-turn conversations and keep context within a session
Content generation	Articles, emails, marketing copy, creative writing usually as a first draft a person then edits
Code generation	Tools like GitHub Copilot writing code from plain-language descriptions, including the notebook attached to this assignment
Summarisation	Cutting long reports and papers down to something quicker to read
Translation and tutoring	Explaining concepts more simply, answering student questions, translating on demand

Beyond that list, GPT-based models also show up in sentiment analysis on customer feedback, generating meeting notes from transcripts, and running as the reasoning layer behind AI agents that can call other tools, search the web, or trigger other software.

6. Python Implementation Example

This example uses Hugging Face's Transformers library to load GPT-2 a smaller, open member of the GPT family and generate a continuation from a prompt. The runnable version, with an interactive loop and a comparison across several prompts, is in the accompanying notebook, Week8_Practical_Tasks.ipynb.

```
from transformers import pipeline
```

```
# Load the pretrained GPT-2 model and tokenizer
generator = pipeline("text-generation", model="gpt2")

# Provide a prompt and ask GPT-2 to continue it
result = generator("Natural Language Processing is",
max_length=50)

print(result[0]["generated_text"])
```

Running this loads GPT-2 and its tokenizer, turns the prompt into numerical tokens, and keeps predicting the next token through Softmax probabilities until it hits the max length. What comes out is a fluent continuation built through the autoregressive process from Section 2. Swap in a different kind of prompt a factual sentence versus something like “Once upon a time” and the style of continuation shifts noticeably, which is a fairly direct demonstration that GPT-2's output comes from patterns in its training data rather than any explicit grammar rules.